



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

**GENESCAN: DNA SEQUENCE ANALYSIS USING NAVIE
STRING MATCHING AND BOYER-MOORE ALGORITHM**

K.Vaishnavi 2520030198

M. Likhitha 2520030575

AVSNR. Akhila 2520030022

COURSE INSTRUCTOR

**Dr. J Sirisha Devi
Professor**

GENESCAN: DNA SEQUENCE ANALYSIS USING NAIVE STRING MATCHING AND BOYER-MOORE ALGORITHM

Introduction

DNA, or Deoxyribonucleic Acid, contains genetic information in the form of nucleotide sequences. These sequences are represented using four basic nucleotide characters: **A (Adenine), C (Cytosine), G (Guanine), and T (Thymine)**. A DNA sequence can contain thousands or millions of bases, and analyzing such sequences often requires searching for specific patterns or motifs.

GeneScan is a DNA sequence analysis application developed to demonstrate how **string matching algorithms from Data Structures and Algorithms** can be applied to biological sequence analysis. The primary purpose of the system is to search for a specific DNA pattern, called a **motif**, within a larger DNA sequence.

The project focuses on two string matching algorithms:

1. **Naive String Matching**
2. **Boyer-Moore String Matching**

The same DNA sequence and the same motif are processed using both algorithms. Their outputs are then compared using measurable parameters such as **number of matches, match positions, character comparisons, shifts, and execution time**.

The project therefore combines a practical application domain, DNA sequence analysis, with fundamental concepts from string searching algorithms

Problem Statement

Searching for a specific DNA pattern manually is difficult when the sequence becomes large. A DNA sequence may contain hundreds, thousands, or even millions of nucleotide characters. A computational technique is therefore required to locate occurrences of a particular motif efficiently and accurately.

For example, consider the DNA sequence:

ATGCGTACGTAGCTAGCTAGCTAGCTAGCTA

and the motif:

GCTAGCTA

The system has to search the complete DNA sequence and determine:

- Whether the motif exists
- How many times the motif occurs
- The positions at which it occurs
- How many character comparisons are required
- How much time is required for the search

Instead of implementing only one searching method, GeneScan uses two different algorithms so that their behavior can be studied and compared.

This makes the project useful not only as a DNA analysis tool but also as a practical demonstration of how algorithm design affects search performance.

Objectives of the Project

The main objective of GeneScan is to implement and demonstrate DNA pattern searching using two string matching algorithms.

The project aims to:

- Accept a DNA sequence from the user.
- Accept a DNA motif or pattern to search for.
- Support DNA input through direct text and supported document formats.
- Validate and normalize the DNA sequence.
- Implement **Naive String Matching** from scratch.
- Implement **Boyer–Moore String Matching** using its searching heuristics.
- Identify all occurrences of the selected motif.
- Record the positions of matching patterns.
- Count the number of character comparisons.
- Measure the execution time of each algorithm.
- Compare the results of Naive and Boyer–Moore.
- Verify that both algorithms produce the same match positions.
- Provide a performance comparison based on the actual input.
- Demonstrate the application of Data Structures and Algorithms concepts in a real-world biological problem.

Algorithms Used

The project implements two algorithms:

- **Naive String Matching**
- **Boyer–Moore String Matching**

Both algorithms search for the same DNA motif so their performance can be compared.

System Workflow

DNA Input



Sequence Extraction



DNA Validation and Normalization



Motif / Pattern Input



Select Algorithm



Naive String Matching

OR

Boyer–Moore String Matching



Search Results



Run Remaining Algorithm



Compare Both Algorithms



Performance Analysis

ALGORITHM / PSEUDO CODE

DNA PREPROCESSING

Algorithm: Extract And Validate DNA

Input: Raw text or uploaded file

Output: Valid DNA sequence

- 1. Read the input.**
- 2. If the input is a file:**
 - a. Detect the file type.**
 - b. Extract its text.**
- 3. Identify the meaningful DNA sequence.**
- 4. Remove unnecessary whitespace.**
- 5. Convert the sequence to uppercase.**
- 6. Ignore FASTA headers beginning with '>'.**
- 7. Check every character.**
- 8. If a character is not A, C, G or T:**
report invalid DNA input.
- 9. Return the validated DNA sequence.**

Algorithm: NaiveSearch

Input:

Text T of length n

Pattern P of length m

Output:

Match positions

Number of comparisons

Execution time

- 1. Initialize an empty list of match positions.**
- 2. Set comparisons = 0.**
- 3. Start execution timer.**
- 4. For i = 0 to n - m:**
 - a. Set j = 0.**
 - b. While j < m:**
 - i. Increment comparisons.**
 - ii. Compare T[i+j] with P[j].**
 - iii. If they are different:**
break.
 - iv. Otherwise increment j.**
 - c. If j == m:**
store i as a match position.
- 5. Stop execution timer.**
- 6. Return all match positions,
comparisons and execution time.**

Algorithm: Boyer- Moore

Build_Bad_Character_Table

Input:

Pattern P of length m

Output:

Bad Character Table

1. Initialize every character's last occurrence as -1.
2. For $i = 0$ to $m - 1$:
Set $\text{LastOccurrence}[P[i]] = i$.
3. Return LastOccurrence table.

Algorithm: Build_Good_Suffix_Table

Input:

Pattern P of length m

Output:

Good Suffix Shift Table

1. Initialize the shift table.
2. Identify suffixes of the pattern.
3. Find matching suffix occurrences within the pattern.
4. Calculate the safe shift for each mismatch position.
5. If a matching suffix is not available,
use the appropriate fallback shift.
6. Return the Good Suffix table.

Algorithm: Boyer_Moore_Search

Input:

DNA sequence T of length n

Pattern P of length m

Output:

Match positions

Number of comparisons

Number of shifts

Execution time

1. Build the Bad Character table.
2. Build the Good Suffix table.
3. Set shift = 0.
4. Initialize MatchPositions.
5. Set comparisons = 0.
6. Set shifts = 0.
7. Start execution timer.
8. While shift $\leq n - m$:
 - a. Set $j = m - 1$.
 - b. Compare the pattern with the DNA
from right to left.
 - c. While $j \geq 0$ and $P[j] == T[\text{shift} + j]$:
comparisons = comparisons + 1
 $j = j - 1$

d. If $j < 0$:

A complete match is found.

Add shift to MatchPositions.

Calculate the next safe shift.

e. Else:

comparisons = comparisons + 1

Calculate Bad Character shift.

Calculate Good Suffix shift.

Choose the larger valid shift.

f. Update shift.

g. shifts = shifts + 1.

9. Stop the execution timer.

10. Return:

Match Positions

comparisons

shifts

execution time

RESULTS

Screenshots:

GeneScan

DNA SEQUENCE ANALYSIS TOOL

Home

Analyze

Boyer-Moore

Comparison

Applications

Start Analysis

BOYER-MOORE STRING MATCHING

1

ALGORITHM

Boyer-Moore

LIVE SEQUENCE

Ready

Analyze DNA.
Find Patterns
Faster.

GeneScan is a DNA sequence analysis platform built around the Boyer-Moore string matching algorithm. Search motifs, inspect matches, measure comparisons and visualize the algorithm.

Start DNA Analysis

Learn Boyer-Moore

01

02

03

04

01

ATGGCTAC

GTAGCTAG

CTAGCTAG

02

GCTAGCAT

GCTAGCTA

GATCGTAC

03

TAGCTAGC

TAGCTAGC

GCTAGCTA

04

CGATCGTA

CGTAGCTA

GATCGATC

✓

PATTERN DETECTED

GCTAGCTA

03
MATCHES

Previous

Play

Next

96.5%

96.5% SPEED IMPROVEMENT vs Naïve Search

01 Analysis

02 Boyer-Moore

03 Naïve

04 Comparison

0 of 2 algorithms completed

Analysis

Provide DNA text or upload a document. GeneScan extracts and previews the validated sequence before search.

DNA SOURCE

PASTE OR TYPE

Paste DNA sequence, multiline DNA, or text containing a DNA sequence...

Or upload a file

Choose File

genescan_..._dna (1).txt

Accepted: TXT, PDF, DOC, DOCX

genescan_sample_dna (1).txt

DNA MOTIF / PATTERN

A/C/G/T

GCTAGCTA

G C T A G C T A

Extract and Validate DNA

✓ Analysis

✓ Boyer-Moore

03 Naïve

04 Comparison

1 of 2 algorithms completed

Boyer-Moore DNA Analysis

Running against the extracted DNA sequence and motif retained from Analysis.

Download highlighted copy

Analysis complete

BOYER-MOORE DNA ANALYSIS

Boyer-Moore Result

Input type: TXT

File: genescan_sample_dna (1).txt

Sequence length: 539 bases

Motif: GCTAGCTA

Algorithm status: Completed

SEQUENCE LENGTH

539

bases

MATCHES FOUND

24

occurrences

CHARACTER COMPARISONS

414

measured checks

SHIFTS

143

pattern mismatches

EXECUTION TIME

6.892900

milliseconds

MATCH POSITIONS

0 indexed

pos 11

pos 15

pos 19

pos 23

pos 46

pos 78

pos 119

pos 123

pos 127

pos 162

pos 203

pos 230

pos 256

pos 291

pos 295

pos 332

pos 364

pos 390

pos 394

pos 424

pos 463

pos 467

pos 486

pos 522

BAD CHARACTER TABLE

Last occurrence

A

7

C

5

G

4

T

6

GOOD SUFFIX TABLE

Shift values

0

1

2

3

4

5

6

7

8

4

4

4

4

4

8

8

8

1

REMAINING ALGORITHM

Naïve String Matching will use the same extracted sequence and motif. No re-upload is needed.

Run Naïve Analysis

Both algorithms found identical match positions.

NAIVE STRING MATCHING

Naive String Matching Result

Download highlighted copy

Analysis complete

Input type: TXT

File: genescan_sample_dna (1).txt

Sequence length: 539 bases

Motif: GCTAGCTA

Algorithm status: Completed

SEQUENCE LENGTH

539

bases

MATCHES FOUND

24

occurrences

CHARACTER COMPARISONS

914

measured checks

SHIFTS

N/A

not applicable

EXECUTION TIME

1.633200

milliseconds

MATCH POSITIONS

0 indexed

#1 11

#2 15

#3 19

#4 23

#5 46

#6 78

#7 119

#8 123

#9 127

#10 162

#11 203

#12 230

#13 256

#14 291

#15 295

#16 332

#17 364

#18 390

#19 394

#20 424

#21 463

#22 467

#23 486

#24 522

RESULTS VERIFIED

Both algorithms found identical match positions.

RESULTS VERIFIED

Both algorithms found identical match positions.

File C:/Users/APPLE/Downloads/genescan_sample_dna%20(1)-highlighted%20(1).html

☆ ☆ 👤 ⋮ Chat

genescan_sample_dna (1).txt

Pattern: GCTAGCTA · 24 matches highlighted

ATGCGTACGTA**GCTAGCTAGCTAGCTAGCTA**ACCGGTTAACGGAT**GCTAGCTA**CGATCGATCGGTACGTTACCGGAT**GCTAGCTA**GGCTTACGATCGATCGTTAGGCATGCAAGCTTA**GCTAGCTAGCTAGCTA**GCTTGCATCGATCGGA
TCCGTAACCGT**GCTAGCTA**AGTCGATCGTACGGTACCGATGCTAACGGTTAA**GCTAGCTA**CGTAGCTTACGGATCGATC**GCTAGCTA**ATGCGTACCGGATTAAACC**GCTAGCTA**GGCTAACGGTATGCGATCGGATCTAAC**GCTAGCTAGCT**
AAGGCTTACGATCGTACGGTCAATACCGAT**GCTAGCTA**GCTGATCGATCGTTAACGGATC**GCTAGCTA**ATCGGTACGATCGGCTAA**GCTAGCTAGCTA**ACGGTATCGATCGTACCGTTA**GCTAGCTA**GCGTATCGATCGGATCAATGC
GTACCGTTAG**GCTAGCTAGCTA**ACCGATCGTAA**GCTAGCTA**GGATCGTACCGTAGCTAACGGTTCGATC**GCTAGCTA**GCTTACGGA

S